

Pandoc Image Rendering Test

Demonstration: Standard Markdown syntax vs. HTML `<img>` tag syntax in Pandoc exports.

Test image: `./figures/SolarSystemPosters_NASA_2130.jpg`

Example A: Standard Markdown Image Syntax

This image is embedded using standard Markdown syntax. Pandoc parses it into a native `Image` node in its AST, so every writer knows how to render it.



Figure 1: Sample Placeholder Image

- **Source:** `! [Alt] (./figures/SolarSystemPosters_NASA_2130.jpg "Title")`
 - **AST node:** `Image ("", [], []) [Str "Sample..."] ("./figures/...", "fig:Standard Markdown Image")`
 - **Expected output:** renders in HTML, PDF, DOCX, EPUB, and LaTeX.
 - **Note:** because the image sits alone in its own paragraph, the `implicit_figures` extension turns it into a figure and promotes the alt text to a caption. Disable with `-f markdown-implicit_figures`.
 - **Limitation:** no way to set width, height, id, or class.
-

Example B: HTML `<img>` Tag Syntax

This image is embedded using an HTML `<img>` tag with custom styling.

- **AST node:** `RawInline` (Format "html") `"<img src=... />"` — **not** an `Image` node.
 - **Expected output in HTML export:** rendered correctly, attributes and inline CSS intact.
 - **Expected output in PDF / DOCX / LaTeX export: silently dropped.** Raw HTML is passed through only to HTML-family writers; all other writers discard `RawInline`/`RawBlock` content whose format they don't recognize.
 - **Clarification:** it is *dropped*, not converted to plain text. You only get literal visible text (`<img src=...>`) if you disable raw HTML parsing entirely with `-f markdown-raw_html`, which escapes the tag instead of storing it as raw HTML.
 - **Diagnostic:** run `pandoc example.md -t native | grep -i rawinline` to see the node Pandoc actually built.
-

Example C: Markdown Image Syntax with Pandoc Attribute Blocks

The recommended fix. Pandoc's `link_attributes` extension lets you attach an identifier, classes, and key/value attributes to a native image — giving you the sizing control of an HTML tag while keeping a real `Image` node that every writer understands.

C1 — Absolute pixel dimensions



Figure 2: Sized with pixels



Figure 3: Sized at 60% of the text width

C2 — Relative width (scales to text block / page width)

C3 — Inline image, no figure or caption

Text flowing around a small inline image  that sits in the middle of a sentence. An image with empty alt text and surrounding text on the same line never becomes a figure.

- **HTML:** ``
- **LaTeX/PDF:** `\includegraphics[width=3.125in,height=1.5625in]{./figures/...}` (px are converted at 96 dpi; use `--dpi=150` to change the assumption)
- **DOCX:** a real embedded picture, sized correctly.
- **Percent widths** become `width="60%"` in HTML and `width=0.6\textwidth` in LaTeX.
- **Styling:** the `.bordered` class reaches HTML and DOCX (as a custom style); inline CSS has no LaTeX equivalent, so borders in PDF need a template or a filter.

Example D: HTML `<img>` Tag Syntax Rescued by a Lua Filter

The same raw HTML as Example B, but this document is converted with `--lua-filter=raw-img.lua`. The filter intercepts `RawInline`/`RawBlock` nodes whose format is `html` and whose text contains an `<img>` tag, re-parses them through Pandoc's HTML reader, and returns genuine `Image` nodes into the AST.

- **Without the filter:** both images vanish from PDF, DOCX, and LaTeX.
- **With the filter:** both survive everywhere, keeping `src`, `alt`, `width`, and `height`.

- **Second case:** the `<p align="center">` wrapper makes the whole thing a `RawBlock`, so the filter must handle `RawBlock` as well as `RawInline` — a common oversight.
- **Lost in translation:** the `style` attribute is carried into HTML output but ignored by LaTeX and DOCX, since inline CSS has no counterpart in those formats.
- **Idempotent for HTML:** the reconstructed `Image` node renders back to an `<img>` tag, so running the filter on HTML output changes nothing meaningful.

Expected Results Matrix

Example	Syntax	HTML	PDF / LaTeX	DOCX	Sizing control
A	<code>![alt](src)</code>	yes	yes	yes	none
B	<code>raw </code>	yes	dropped	dropped	HTML only
C	<code>![alt](src){...}</code>	yes	yes	yes	px, in, cm, %
D	<code>raw + Lua filter</code>	yes	yes	yes	px, % via filter

Verification Commands

Inspect the AST - see Image vs RawInline for yourself

```
pandoc example.md -t native | head -60
```

Baseline conversions (Examples B and D images will be missing from PDF/DOCX)

```
pandoc example.md -s -o out/test-baseline.html
```

```
pandoc example.md -s -o out/test-baseline.pdf
```

```
pandoc example.md -s -o out/test-baseline.docx
```

With the filter (all four examples survive)

```
pandoc example.md -s --lua-filter=raw-img.lua -o out/test-filtered.pdf
```

```
pandoc example.md -s --lua-filter=raw-img.lua -o out/test-filtered.docx
```

Show the escaping behaviour when raw_html is disabled

```
pandoc example.md -f markdown-raw_html -t html | grep -i "&lt;img"
```

Single self-contained HTML with images base64-inlined

```
pandoc example.md -s --embed-resources --lua-filter=raw-img.lua -o out/test-embedded.html
```

If your images live outside the working directory, add `--resource-path=.:figures:assets` so Pandoc can resolve the relative paths.